

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Simulação de um Escalonador de Processos

BCC202 - Estrutura de Dados 1

Pedro Henrique Menezes Féo de Castro e Daniel Matos Falcão
Professora: Karla Alexsandra de Souza Joriatti

Ouro Preto
3 de fevereiro de 2026

Sumário

1	Introdução	1
1.1	Especificações do problema	1
1.2	Considerações iniciais	1
1.3	Ferramentas utilizadas	1
1.4	Instruções de compilação e execução	1
2	Desenvolvimento	2
2.1	Tipo Abstrato de Dados (TAD)	2
2.2	Algoritmos de Ordenação	2
2.3	Funcionamento do Escalonador	2
2.4	Incluindo fragmento de códigos	2
3	Resultados	4
3.1	Complexidade Temporal	4
3.2	Complexidade Espacial	4
3.3	Discussão dos Resultados	4
4	Considerações Finais	5

Lista de Códigos Fonte

1	Interface do TAD FilaProcessos (filaprocessos.h).	2
2	Implementação da Solução (filaprocessos.c).	3

1 Introdução

Este trabalho consiste na simulação de um escalonador de processos de um sistema operacional. O objetivo é gerenciar o acesso à CPU com base em prioridades, ordem de chegada e um limite de ciclos de execução fixado em $X = 100$.

1.1 Especificações do problema

O escalonador opera sob a regra de atender prioritariamente os processos de maior prioridade e, em caso de empate, segue a ordem de chegada. Além disso, aplica-se um sistema de ciclos: processos não podem exceder 100 ciclos de CPU em uma única rodada.

1.2 Considerações iniciais

Algumas ferramentas foram utilizadas durante a criação deste projeto:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code + Extensão Live Share.
- Linguagem utilizada: C.
- Ambiente de desenvolvimento da documentação: Overleaf $\text{\LaTeX}$.

1.3 Ferramentas utilizadas

Algumas ferramentas foram utilizadas para testar a implementação, como:

- *Valgrind*: ferramentas de análise de vazamento de memória.

1.4 Instruções de compilação e execução

Para a compilação do projeto, basta digitar:

Compilando o projeto

```
gcc -c filaprocessos.c -Wall
```

```
gcc -c tp.c -Wall
```

```
gcc filaprocessos.o tp.o -o exe -lm
```

Usou-se para a compilação as seguintes opções:

- *-Wall*: para mostrar todos os possíveis *warnings* do código.

Para a execução do programa basta digitar:

```
./exe < casoteste.in
```

2 Desenvolvimento

2.1 Tipo Abstrato de Dados (TAD)

Foi construído um TAD para representar o **Processo** e a **FilaProcessos**

- **Processo**: Contém ID, tempo de chegada (T), prioridade (P) e ciclos (C).
- **FilaProcessos**: Implementada como um vetor dinâmico para suportar as operações de inserção. Também contém o tamanho atual do vetor dinâmico (TAM)

As funções **criaFila**, **destroiFila** e **adicionaFila** foram implementadas para gerenciar a alocação de memória e o preenchimento da fila a partir da entrada.

2.2 Algoritmos de Ordenação

O projeto utiliza dois métodos de ordenação distintos para otimizar o escalonamento:

1. **Merge Sort**: Utilizado exclusivamente para a ordenação inicial dos processos que chegam no mesmo intervalo de tempo. Este método garante a estabilidade ao comparar a prioridade e, em caso de empate, o tempo de chegada.
2. **Insertion Sort**: Aplicado durante a fase de escalonamento. Sempre que um processo é reinserido na fila após exceder os 100 ciclos, sua prioridade é incrementada e o *Insertion Sort* o reposiciona corretamente, levando em conta a nova prioridade.

2.3 Funcionamento do Escalonador

O escalonador opera em um ciclo **while** até que a fila esteja vazia. Em cada iteração:

- O processo no topo da fila (índice 0) é selecionado.
- Ele executa por até 100 ciclos ou até o término de seus ciclos totais.
- Se o processo ainda possuir ciclos pendentes, sua prioridade é incrementada em 1 e ele é reinserido no fim da fila antes da reordenação.
- O log é impresso seguindo o padrão: ID Prioridade CiclosExecutados.

2.4 Incluindo fragmento de códigos

O Código 2 apresenta a interface do **TAD FilaProcessos**.

```
1 #ifndef _FILAPROCESSOS
2 #define _FILAPROCESSOS
3
4 #define CICLOS_MAX 100
5
6 typedef struct filaProcessos FilaProcessos;
7 typedef struct processo Processo;
8
9 FilaProcessos* criaFila(int n);
10 void adicionaFila(FilaProcessos* fila, int i);
11 void destroiFila(FilaProcessos** fila);
12 void escalonador(FilaProcessos* fila);
13 void imprimeLog(Processo p);
14
15 void insertionSort(FilaProcessos* fila, int very);
16 void moveAll(FilaProcessos *fila, int n, int j);
17 void mergeSort(FilaProcessos *fila, int n);
18
19 #endif
```

Código 1: Interface do TAD FilaProcessos (**filaprocessos.h**).

O Código 2 apresenta o escalonador.

```
1 // criaFila , destroiFila , adicionaFila , imprimeLog
2 // omitidas por brevidade
3
4 void escalonador(FilaProcessos* fila) {
5     int ciclos = 0;
6     Processo chave;
7
8     // chama o Merge Sort apenas na primeira iteracao
9     mergeSort(fila, fila->tam);
10
11     // Enquanto ainda houver processos
12     while(fila->tam > 0){
13         chave = fila->processos[0];
14
15         // executa 100 ciclos ou ate os ciclos do processo acabarem
16         while (ciclos < 100 && chave.ciclos > 0) {
17             chave.ciclos--;
18             ciclos++;
19         }
20
21         filaRemove(fila, 0);
22
23         Processo temp = chave;
24         temp.ciclos = ciclos;
25         imprimeLog(temp);
26
27         // Se o processo ainda possui ciclos, incrementa um em sua prioridade
28         // e o re-adiciona na fila e ordenamos novamente usando Insertion
29         // Sort
30         if (chave.ciclos > 0) {
31             chave.prioridade++;
32             fila->processos[fila->tam] = chave;
33             fila->tam++;
34             insertionSort(fila);
35         }
36
37         ciclos = 0;
38     }
39 }
```

Código 2: Implementação da Solução (**filaprocessos.c**).

3 Resultados

3.1 Complexidade Temporal

A complexidade temporal é dominada pela ordenação inicial da fila e pelas sucessivas reordenações necessárias após cada rodada de execução, quando o processo não termina dentro do limite estabelecido.

A ordenação inicial, realizada através do algoritmo **Merge Sort**, ocorre em tempo $O(N \log N)$, onde N representa a quantidade total de processos. Esta etapa garante que a simulação comece com a fila organizada por prioridade e, em caso de empate, pelo tempo de chegada.

Durante o ciclo do escalonador, cada processo que excede o limite de 100 ciclos de CPU é reinserido na fila com sua prioridade incrementada em 1. A cada reinserção, o algoritmo utiliza o **Insertion Sort** para manter a ordem das prioridades. Conforme a lógica do código, a reordenação e a manutenção da fila (incluindo a remoção e deslocamento de elementos) resultam em uma complexidade de $O(N)$ por operação de ajuste.

Uma vez que a maior complexidade temporal do algoritmo refere-se ao algoritmo Merge Sort, a complexidade temporal é:

$$O(N \log N)$$

Este desempenho confirma que o simulador é eficiente para processar o conjunto de processos fornecido, mantendo a integridade das prioridades durante toda a execução.

3.2 Complexidade Espacial

A complexidade espacial é determinada pelo armazenamento da estrutura de dados da fila e pelos vetores auxiliares de manipulação dos processos.

- O armazenamento dos N processos no vetor dinâmico dentro do TAD **FilaProcessos** requer $O(N)$ espaço.
- O algoritmo **Merge Sort** utiliza vetores temporários durante a fase de intercalação, demandando $O(N)$ espaço adicional temporário.

A complexidade espacial total do algoritmo é, portanto:

$$O(N)$$

Este resultado indica que o uso de memória cresce de forma linear em relação ao número de processos de entrada.

3.3 Discussão dos Resultados

Os testes de validação confirmaram o funcionamento correto do algoritmo, demonstrando que o escalonador atende corretamente os processos seguindo a ordem de prioridade e, em casos de empate inicial, a ordem de chegada. O log gerado pelo programa refletiu precisamente a execução dos ciclos de CPU e a interrupção de processos que excederam o limite de 100 ciclos. A análise com a ferramenta *Valgrind* confirmou a ausência de *memory leaks*, validando que toda a memória alocada para o TAD **FilaProcessos** e seus elementos foi devidamente liberada ao final da execução. A eficiência obtida com a combinação dos algoritmos **Merge Sort** e **Insertion Sort** mostrou-se superior à abordagens de busca exaustiva, permitindo que o escalonamento de uma grande carga de processos ocorra sem degradação perceptível de desempenho.

4 Considerações Finais

O desenvolvimento deste trabalho prático proporcionou um aprofundamento essencial nos conceitos de **Algoritmos de Ordenação**, aplicados à simulação de uma função crítica do Kernel de um sistema operacional.

Processo de Implementação e Dificuldades: O maior desafio residiu na implementação da lógica de reinserção ordenada na fila. Garantir que o processo, após executar seus ciclos e ter sua prioridade incrementada, voltasse para a posição correta utilizando o *Insertion Sort* exigiu atenção especial à manipulação dos índices do vetor dinâmico. Além disso, a implementação do *Merge Sort* para a ordenação inicial, permitiu consolidar conhecimentos sobre o modelo de divisão e conquista.

Conclusão: O trabalho prático alcançou seu objetivo de implementar um simulador funcional de escalonador de processos. O principal aprendizado foi a percepção prática de como a escolha de estruturas de dados e algoritmos de ordenação adequados impacta diretamente na organização e eficiência de sistemas que lidam com múltiplas tarefas simultâneas.

Referências

1. PEDRO HENRIQUE L. SILVA **Aulas sobre: Alocação Dinâmica, TAD, Recursão, Análise de Complexidade de Algoritmos, Métodos de Ordenação e Filas**, BCC202 Estruturas de Dados I. Universidade Federal de Ouro Preto - UFOP.